

Day 3 exactly step-by-step,

Important for Students: We will **not jump ahead**. Connect day2 to day3

Day 3 syllabus is:

Session	Topic	Time
Session 1	DAX Operations	2 hrs
Session 2	Advanced DAX	2 hrs
Session 3	Hands-on + Q&A	2 hrs

The existing HR workbook explicitly covers basic DAX such as DIVIDE, measures, relationships and bands, so Day 3 should now move into **filter context, CALCULATE, FILTER, RANKX and time intelligence**, rather than repeating those exercises.

The separate HR Project requirement is suitable for these advanced calculations: active headcount, selected-period attrition, department ranking, year-over-year leave change and training-cost analysis

DAY 3 — START HERE

Session 1 — DAX Operations

Step 1: Understand Filter Context

Time: 30 minutes

This should be the **first concept of Day 3**.

On Day 2, we created measures such as:

Total Headcount = COUNTA('EmployeeMaster'[EmployeeID])

and:

Attrition Rate = DIVIDE([Total Exits], [Total Headcount], 0)

A measure can return different results depending on the filters applied to the report.

Suppose Total Headcount = 40.

Without any filter:

Total Headcount : 40

If we select **Sales** in the Department slicer:

Department = Sales → Total Headcount → 8

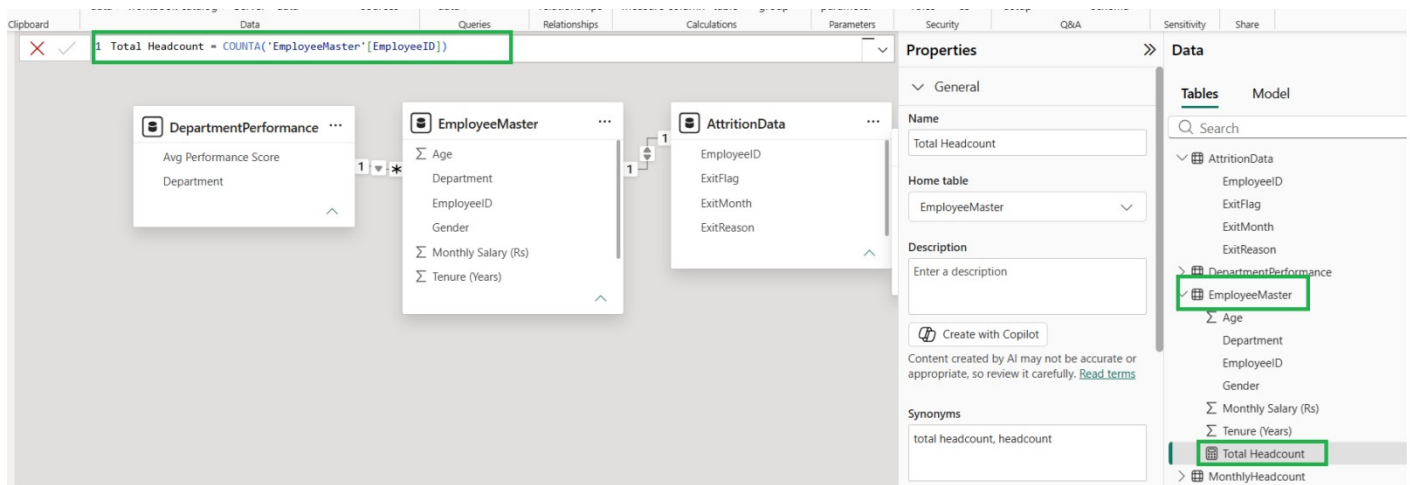
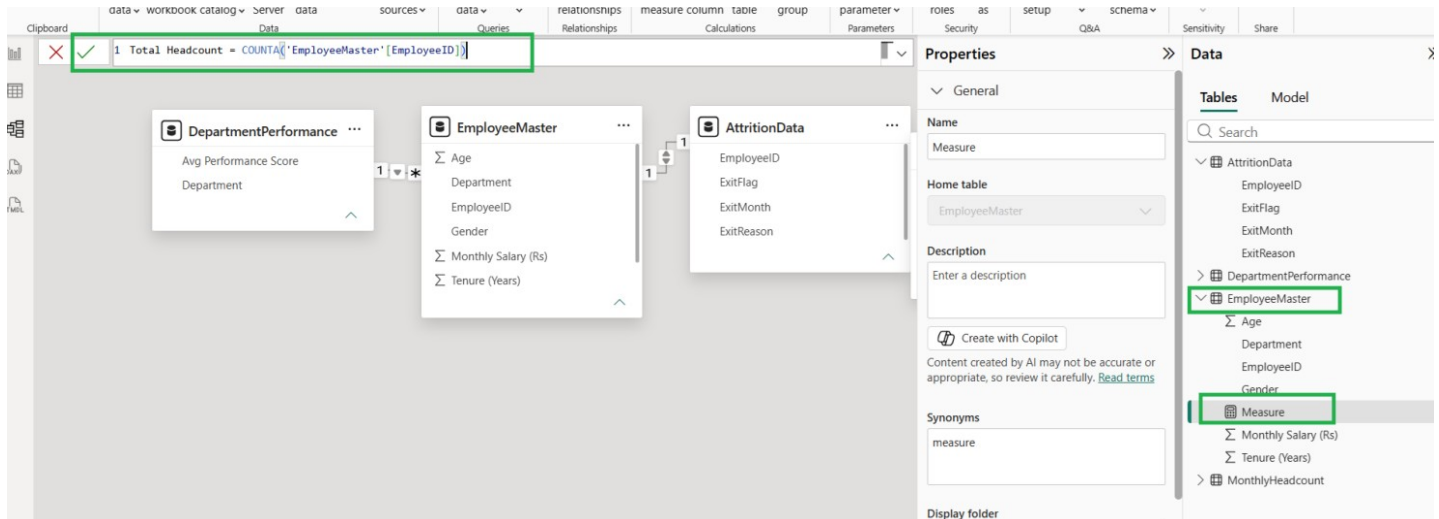
The DAX formula **did not change**.

The **filter context** changed.

Step 1A — Create a simple measure

Modeling → **New measure**

Total Headcount = COUNTA('EmployeeMaster'[EmployeeID])



You already have this measure from Day 2.

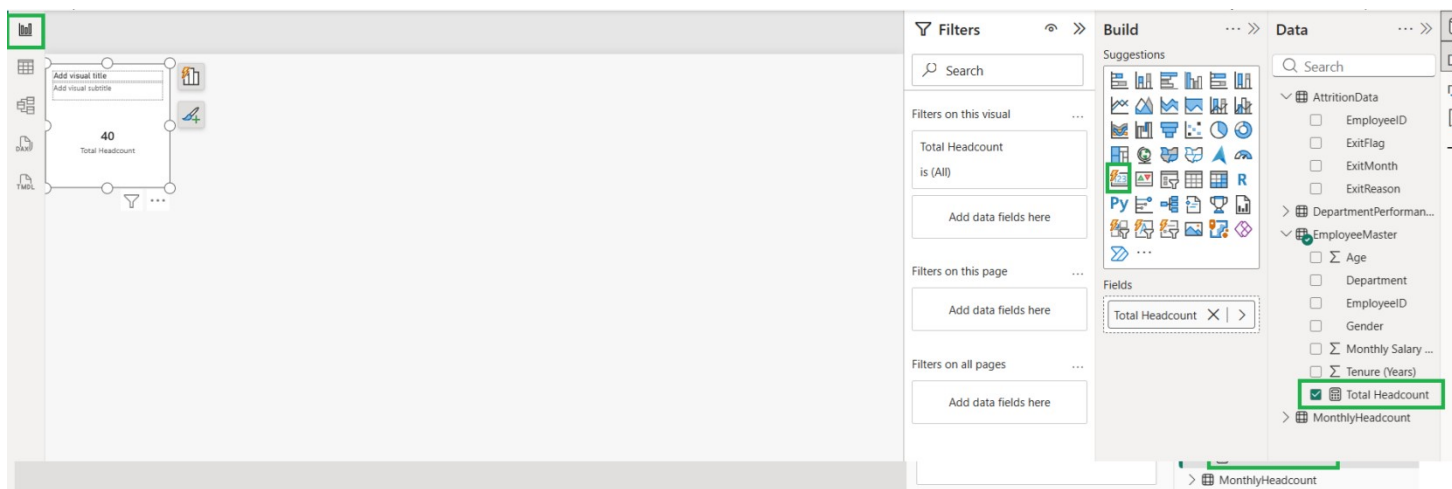
Now create a **Card**.

Put: Total Headcount

into the Card.

You should see:

40

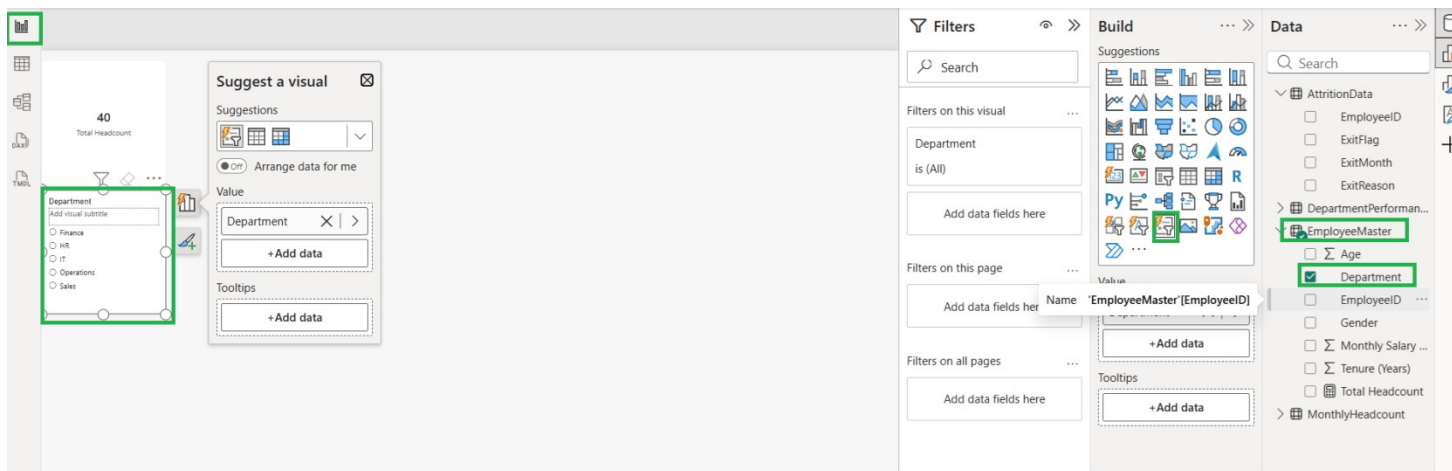


Step 1B — Add Department slicer

Add a **Slicer**.

Put:

EmployeeMaster → Department into the slicer.

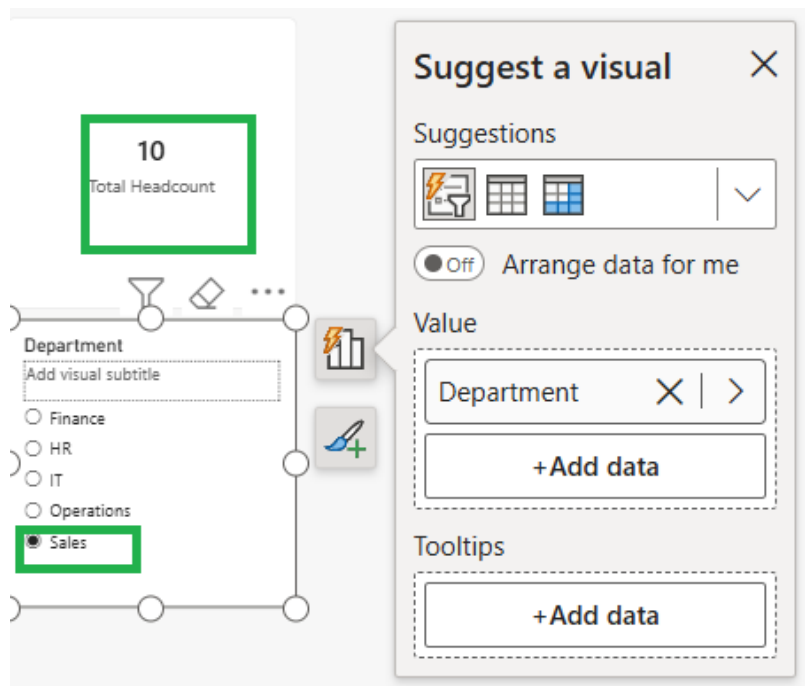


Select:

Sales

Now look at the Card.

The number should change to the number of Sales employees.



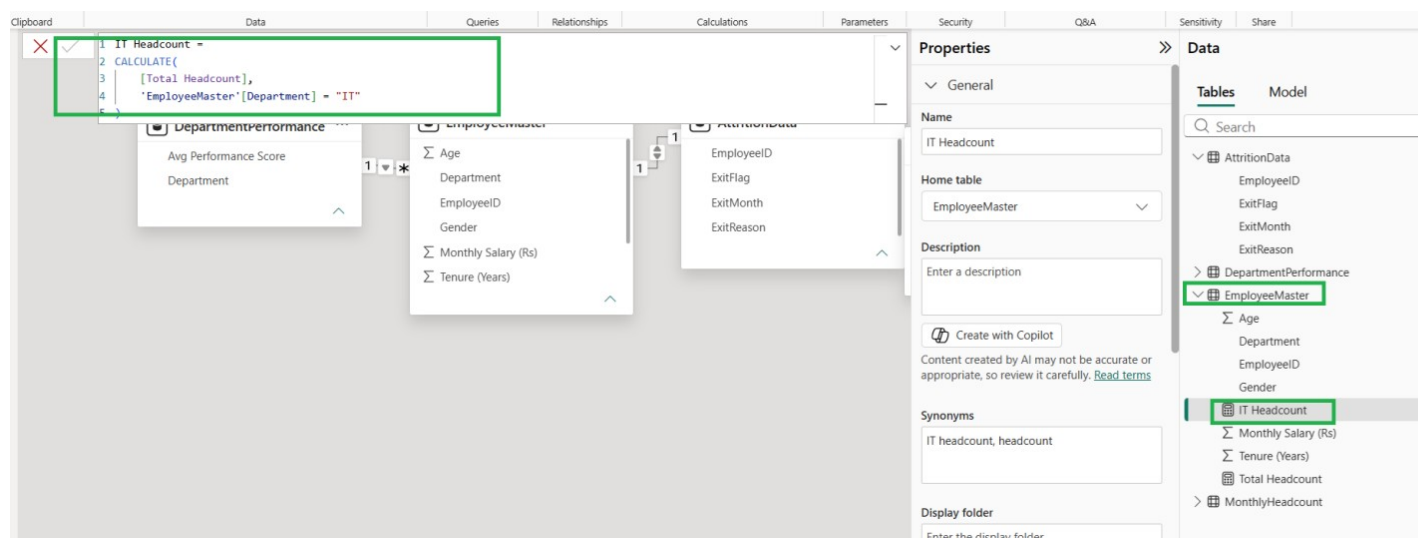
"The DAX measure is the same. Power BI evaluates the measure according to the current filter context."

This is the foundation for everything we will do today.

Step 1C — Introduce CALCULATE

Now create the first **new Day 3 measure**.

```
IT Headcount =  
CALCULATE(  
    [Total Headcount],  
    'EmployeeMaster'[Department] = "IT"  
)
```

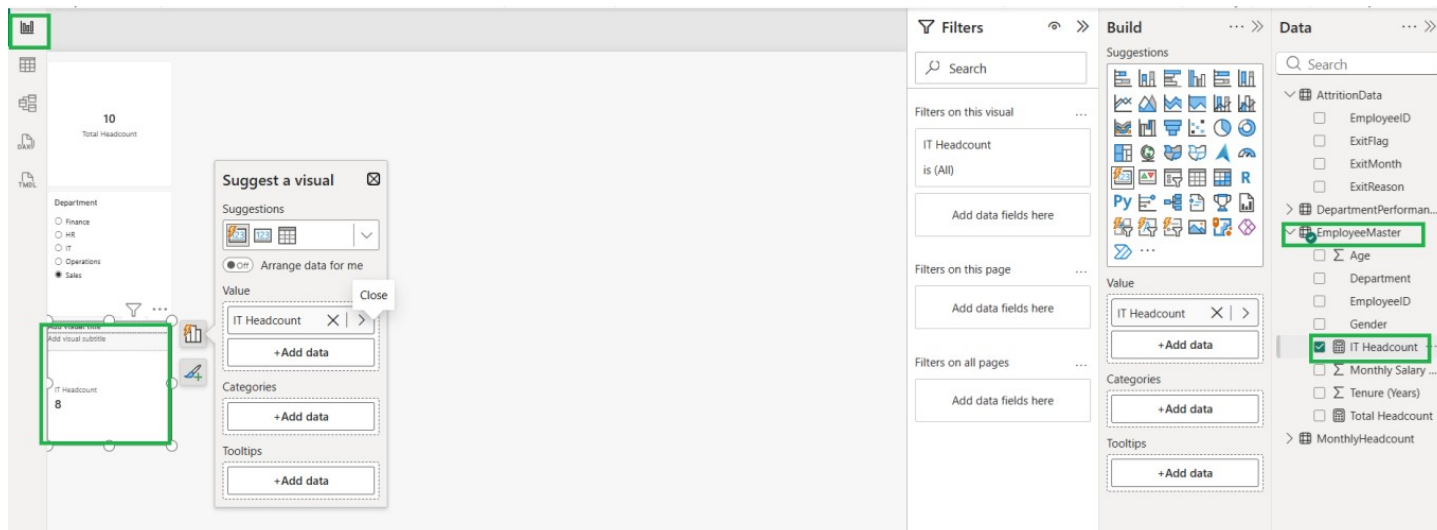


Put this measure into a Card.

It will always calculate:

Headcount for IT

even if the report has other filters.



The **IT Headcount stays 8** because CALCULATE is explicitly applying Department = "IT".

CALCULATE modifies the filter context in which a measure is evaluated.

This is the most important Day 3 concept.

Step 1D — Compare the two measures

Create two Cards:

Measure	Meaning
Total Headcount	Responds to the current report filters
IT Headcount	Applies an IT filter through CALCULATE

This is a very good live demonstration because students can change the Department slicer and observe how measures behave.

Step 1E — First Day 3 exercise

Student's task:

Exercise 1 — Department Headcount

Create:

HR Headcount =

```
CALCULATE(  
    [Total Headcount],  
    'EmployeeMaster'[Department] = "HR"  
)
```

Finance Headcount =

```
CALCULATE(  
    [Total Headcount],  
    'EmployeeMaster'[Department] = "Finance"  
)
```

Create three Cards:

Total Headcount

HR Headcount

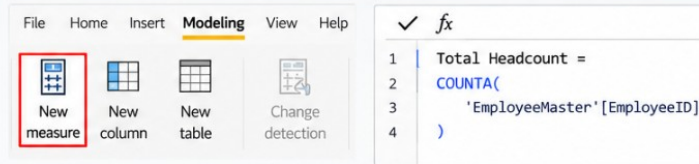
Finance Headcount

Students should understand:

Measure → Filter → Result

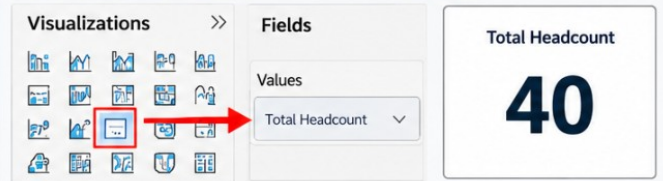
1 Create a basic measure – Total Headcount

1. In the Fields pane, select **EmployeeMaster** table.
2. Go to **Modeling** → **New measure**.
3. Enter the DAX formula and press Enter.



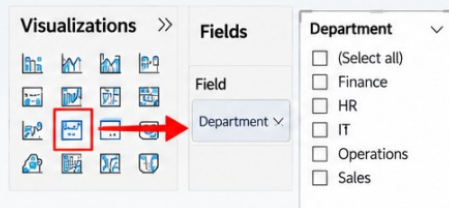
2 Add a Card visual to show Total Headcount

1. From the **Visualizations** pane, select **Card**.
2. Drag the **Total Headcount** measure to the field well.



3 Add a Department slicer

1. Select **Slicer** visual.
2. Drag **Department** from **EmployeeMaster** to the field well.



4 See the effect of filter context

When no filter is selected, the card shows total headcount (40).

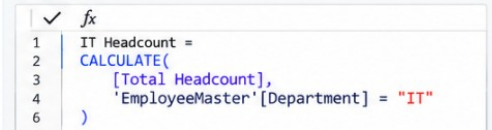


When you select **Sales** in the slicer, the same measure returns the headcount for Sales only.



5 Create a new measure using CALCULATE

1. Go to **Modeling** → **New measure**.
2. Enter the DAX formula.



6 Add multiple cards to compare



Key Takeaways

- ✓ A measure returns different results based on the current filter context.
- ✓ Slicers, visuals and page filters create the filter context.
- ✓ CALCULATE allows you to modify the filter context inside a measure.
- ✓ The DAX formula does not change, only the filter context changes.

DAX Formulas Used

Measure Name	DAX Formula	Purpose
Total Headcount	COUNTA('EmployeeMaster'[EmployeeID])	Counts all employees (respects slicers)
IT Headcount	CALCULATE([Total Headcount], 'EmployeeMaster'[Department] = "IT")	Always returns headcount for IT department
HR Headcount	CALCULATE([Total Headcount], 'EmployeeMaster'[Department] = "HR")	Always returns headcount for HR department
Finance Headcount	CALCULATE([Total Headcount], 'EmployeeMaster'[Department] = "Finance")	Always returns headcount for Finance department

What we have covered so far

Day 2

SUM

AVERAGE

COUNT

COUNTA

COUNTROWS

DIVIDE

Basic measures

Day 3 — Step 1

FILTER CONTEXT



CALCULATE



Modify filter context



Business-specific measures

This is the correct progression.

Next step

After students understand this, **Step 2 will be FILTER()**, where we move from a simple condition like:

Department = "IT"

to more powerful conditions such as:

Salary > 70000

Tenure > 5

Age > 40

Then we'll move into **CALCULATE + FILTER**, followed by **ALL**, **RANKX**, and finally **Time Intelligence**.

We should proceed **one step at a time**, so don't create the advanced measures yet

Day 3 - Step 2: FILTER()

Now we move from “**change the filter context**” to “**apply a condition to a table.**”

Step 2A — Direct filter vs FILTER()

We already have:

IT Headcount =

```
CALCULATE(  
    [Total Headcount],  
    'EmployeeMaster'[Department] = "IT"  
)
```

This is a **simple/direct filter**.

Now let's create a more useful HR example.

Step 2B — Count employees earning more than ₹70,000

Create a new measure:

High Salary Employees =

```
CALCULATE(  
    [Total Headcount],  
    FILTER(  
        'EmployeeMaster',  
        'EmployeeMaster'[Monthly Salary (Rs)] > 70000  
    )  
)
```

The screenshot displays the Power BI interface with two panes: 'Tables' and 'Model'. In the 'Tables' pane, the 'EmployeeMaster' table is selected and highlighted with a green box. In the 'Model' pane, the 'High Salary Employees' measure is created and highlighted with a green box. The 'General' tab is active in the left pane, showing fields for Name, Description, Synonyms, Row label, and Is hidden.

General

Name
EmployeeMaster

Description
Enter a description

Synonyms
employee master, EmployeeMaster, master

Row label
Select a row label

Is hidden
☐ No

Advanced

Tables **Model**

Search

AttritionData
EmployeeID
ExitFlag
ExitMonth
ExitReason

DepartmentPerformance

EmployeeMaster
Age
Department
EmployeeID
Gender
High Salary Employees
IT Headcount
Monthly Salary (Rs)
Tenure (Years)
Total Headcount

MonthlyHeadcount

What is happening?

EmployeeMaster table



FILTER()



Keep employees where

Salary > ₹70,000



CALCULATE()



[Total Headcount]



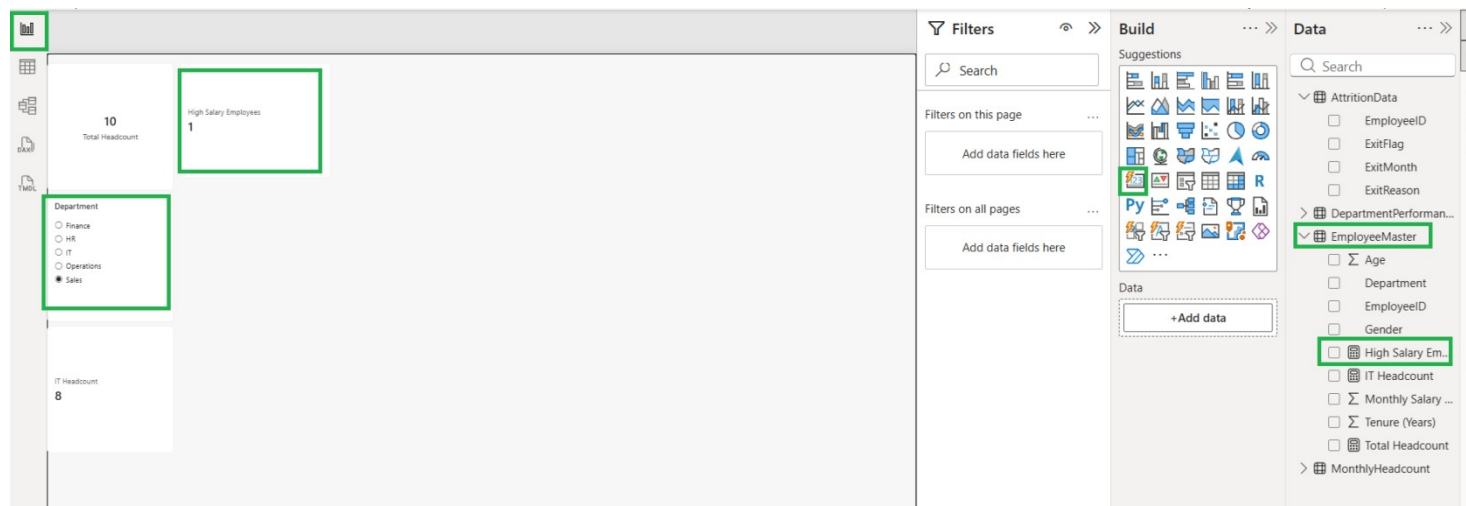
Number of high-salary employees

Step 2C — Test it with your Department slicer

Keep your **Department slicer** on the page.

Try:

Slicer	High Salary Employees
All Departments	Result
Sales	Result
IT	Result
HR	Result
Finance	Result



Notice that the **Department slicer still affects the result**, while **FILTER()** adds the salary condition.

That's an important difference from our earlier IT Headcount example.

Key concept to tell participants

FILTER() returns a filtered table based on a condition.

And:

CALCULATE() evaluates a measure using that modified filter context.

One more quick example

Create:

Senior Employees =

CALCULATE(

[Total Headcount],

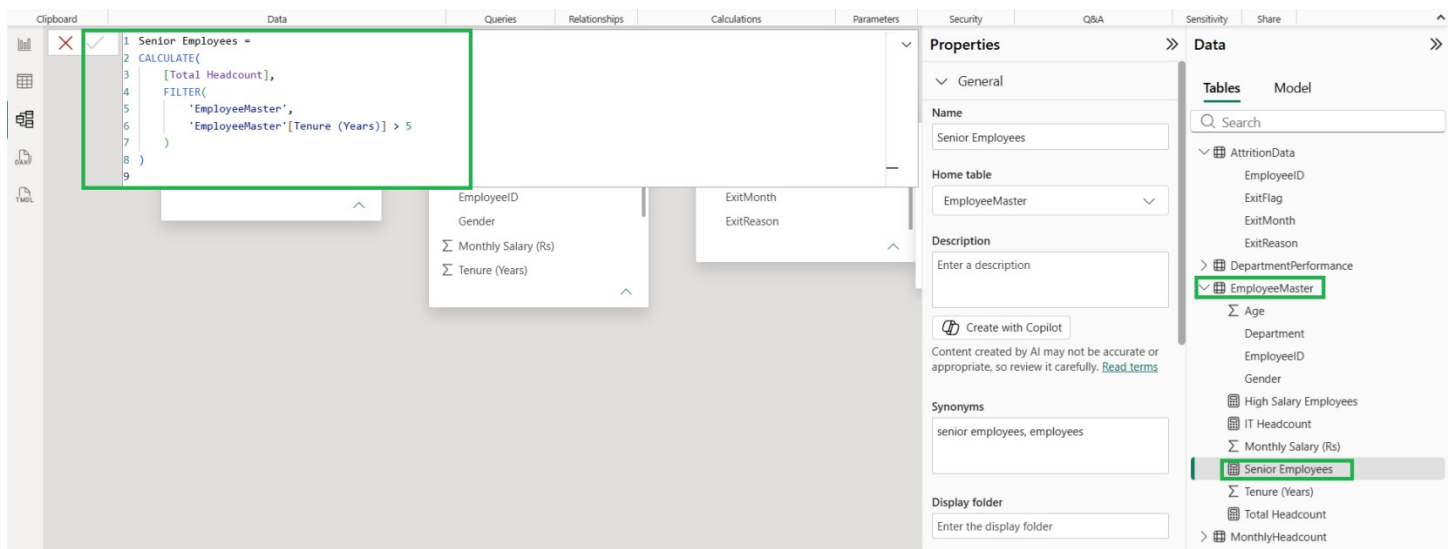
FILTER(

'EmployeeMaster',

'EmployeeMaster'[Tenure (Years)] > 5

)

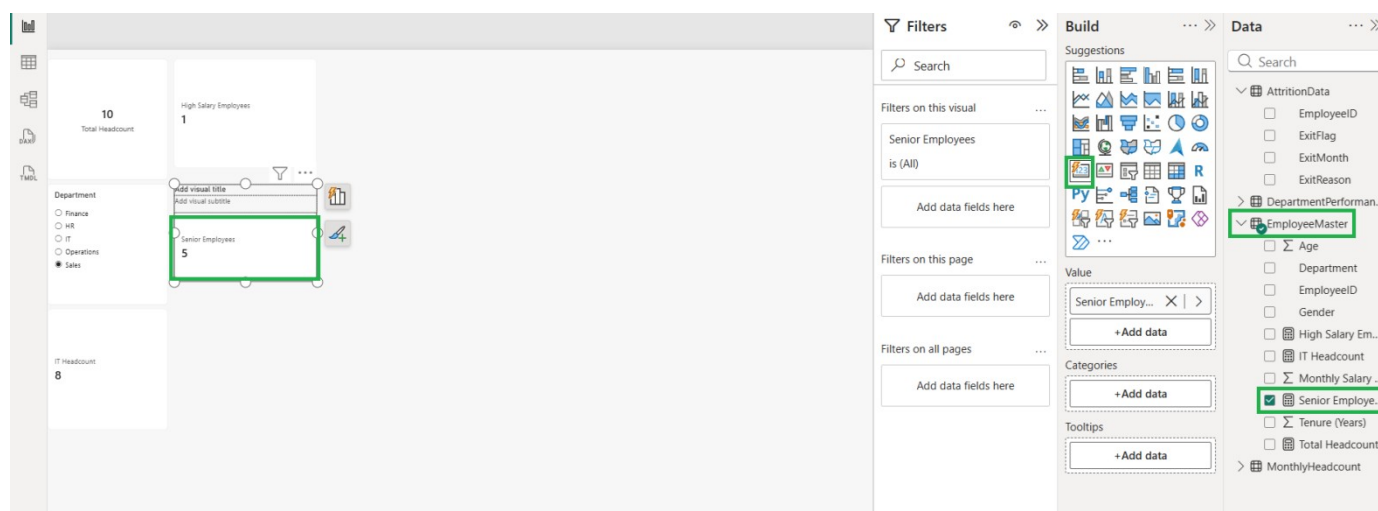
)



Now the measure means:

“How many employees have more than 5 years of tenure?”

Try the Department slicer again.



Q: Why do we need FILTER() when CALCULATE can already take a filter?

Expected answer:

Simple conditions can be written directly in CALCULATE. FILTER() is useful when we need to evaluate a condition row-by-row over a table, especially for more complex filtering logic.

=====

Day 3 - Step 3: ALL()

Now we learn **how to remove a filter**.

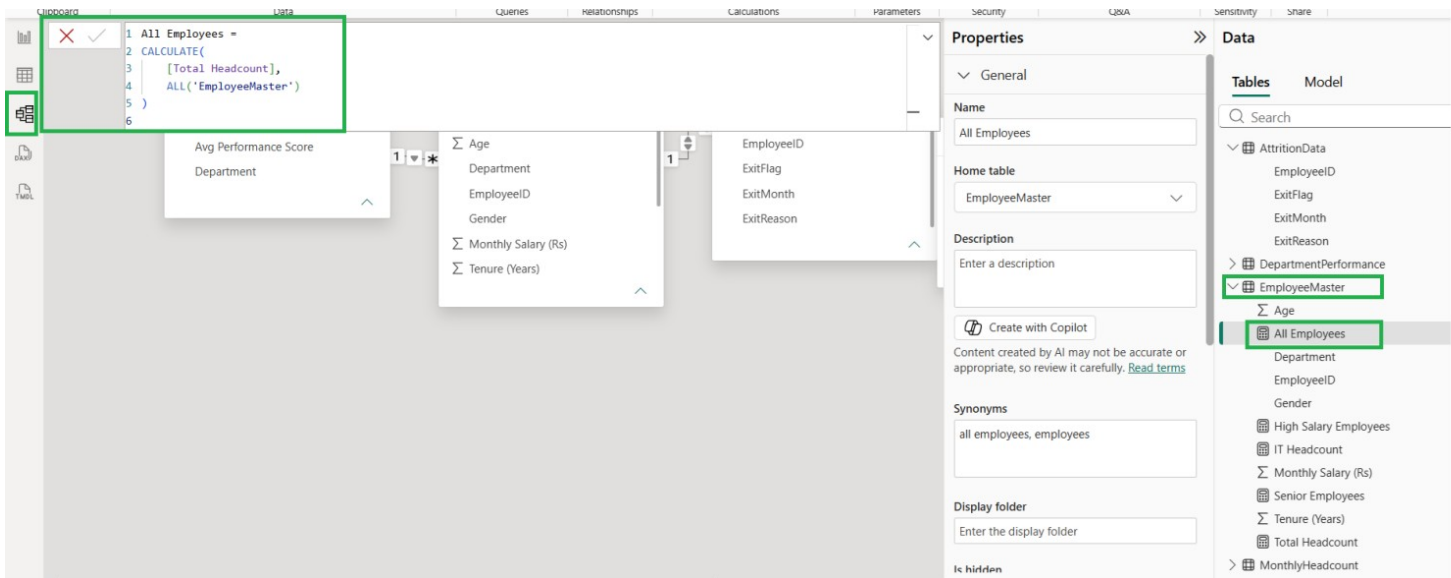
This is especially important for HR dashboards because we often need:

Department Headcount ÷ Total Company Headcount

Step 3A — Create All Employees

Create a new measure:

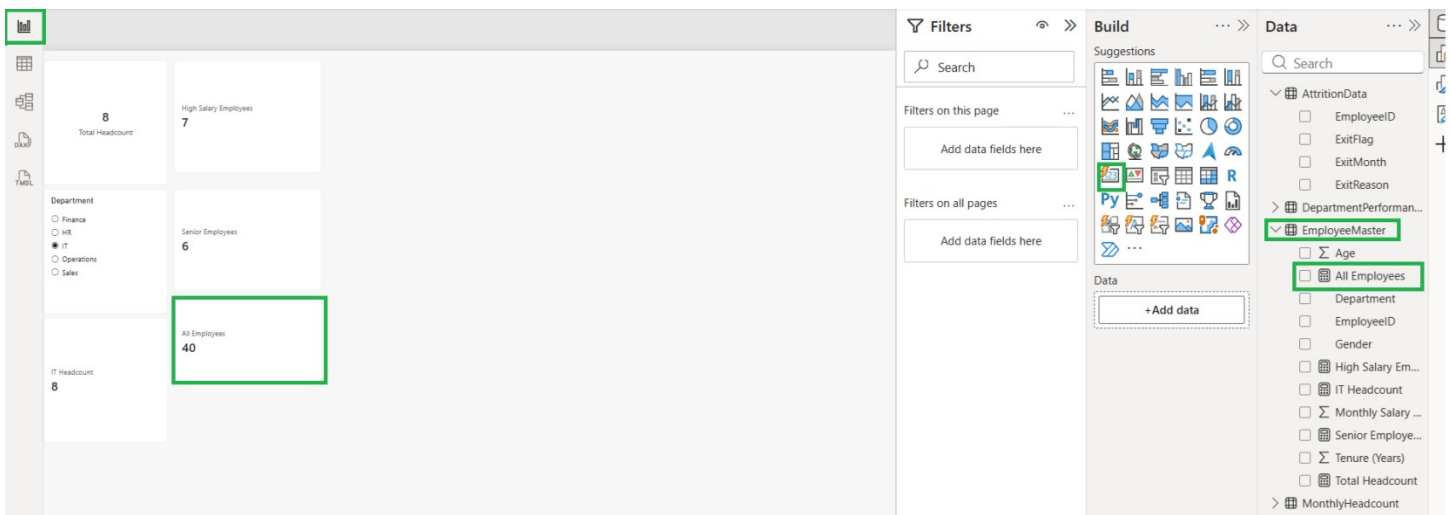
```
All Employees =  
CALCULATE(  
    [Total Headcount],  
    ALL('EmployeeMaster')  
)
```



Add a **Card** for All Employees.

You should see:

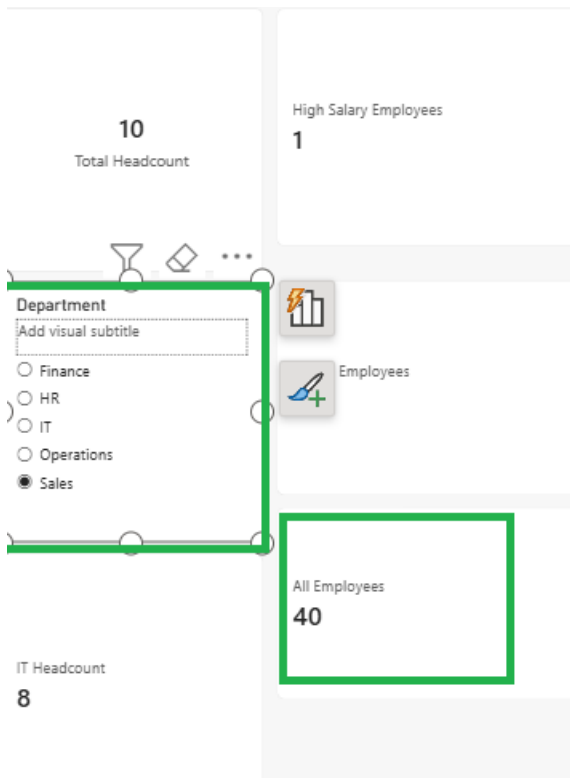
40



Now select **Sales** in your Department slicer.

Notice:

- Total Headcount → **10**
- All Employees → **40**



Why?

The slicer creates:

Department = Sales

But:

ALL('EmployeeMaster')

removes the filters from the EmployeeMaster table.

So:

Sales filter



Total Headcount → 8

Sales filter



ALL(EmployeeMaster)



Filter removed



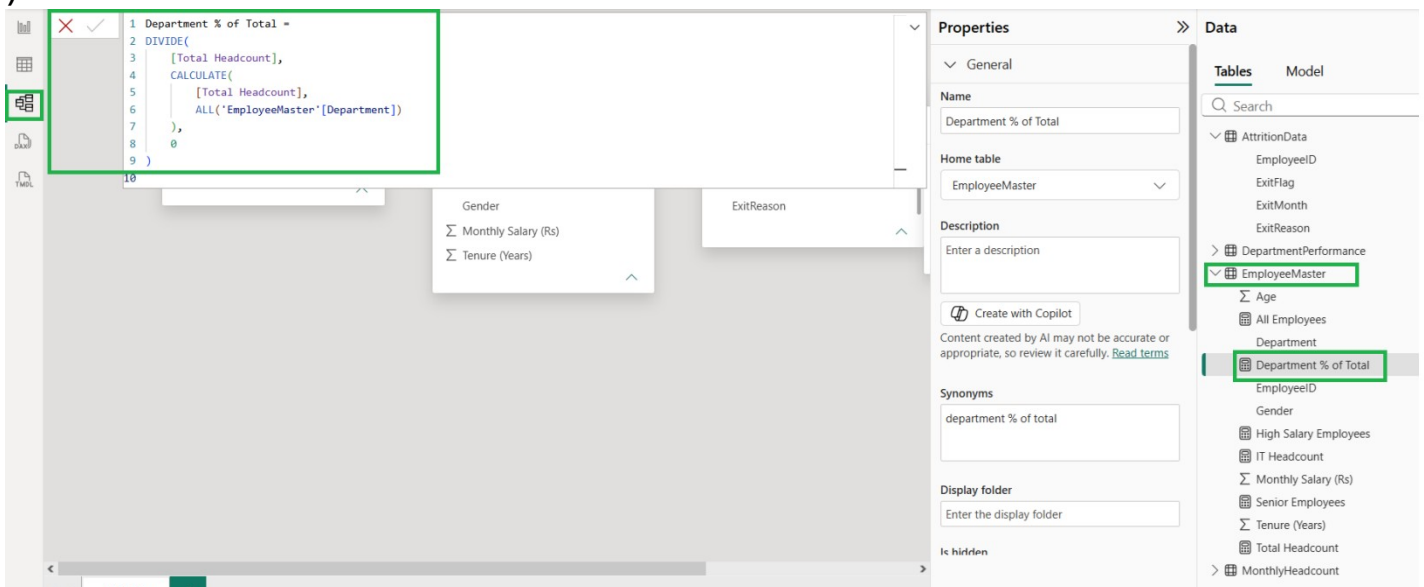
All Employees → 40

Step 3B — Department % of Total

This is the **real business use case**.

Create:

```
Department % of Total =  
DIVIDE(  
    [Total Headcount],  
    CALCULATE(  
        [Total Headcount],  
        ALL('EmployeeMaster'[Department])  
    ),  
    0  
)
```



Format the measure as **Percentage**.

Test with slicer

If Sales has 8 employees:

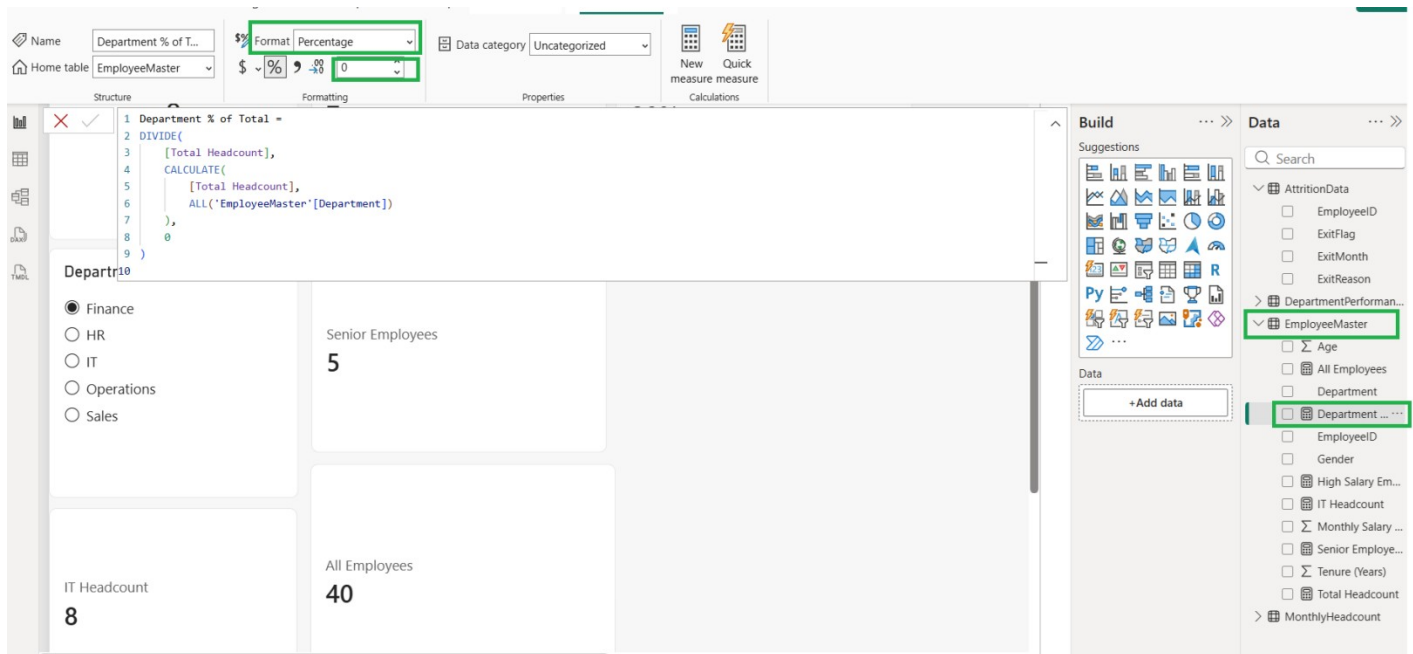
Sales Headcount = 10

Total Company Headcount = 40

$$10 / 40 = 25\%$$

So when you select Sales:

Department % of Total = 25%



Try different departments

Department	Headcount	% of Total
Sales	10	25%
HR	6	15%
Finance	8	20%
IT	8	20%

The exact percentage will depend on your current dataset.

3A All Employees (Ignore Slicer)

ALL() removes the filter from the specified table/column. Here we remove the Department filter so that the **measure always returns** the total company headcount (40) even when a department is selected in the slicer.

1 Create the measure

```

1 All Employees =
2 CALCULATE(
3     [Total Headcount],
4     ALL('EmployeeMaster')
5 )

```

- [Total Headcount] is the measure we created earlier.
- ALL('EmployeeMaster') removes all filters from the EmployeeMaster table, including the Department slicer filter.

2 Add visuals and test with slicer**No Department Selected (All)**

Department
☐ Finance
☐ HR
☐ IT
☐ Operations
☐ Sales

Total Headcount
(respects slicer)
40

All Employees
(ignores slicer)
40

Department	Headcount
Finance	8
HR	6
IT	8
Operations	8
Sales	10
Total	40

Sales Selected in Slicer

Department
☐ Finance
☐ HR
☐ IT
☐ Operations
☒ Sales

Total Headcount
(respects slicer)

10

All Employees
(ignores slicer)

40

Department	Headcount
Sales	10
Total	10

Key Takeaways (3A)

- ✓ ALL() removes the Department filter from the table.
- ✓ All Employees shows 40 even when Sales is selected in the slicer.
- ✓ Total Headcount shows 10 because it respects the slicer.
- ✓ This is useful when you want to show company total next to a filtered value.

3B Department % of Total

We calculate the percentage of employees in the selected department out of the total company headcount. We use ALL('EmployeeMaster'[Department]) to remove only the Department filter.

1 Create the measure

```

1 Department % of Total =
2 DIVIDE(
3     [Total Headcount],
4     CALCULATE(
5         [Total Headcount],
6         ALL('EmployeeMaster'[Department])
7     ),
8     0
9 )

```

- The numerator ([Total Headcount]) respects the slicer.
- The denominator removes only the Department filter using ALL('EmployeeMaster'[Department]).
- DIVIDE is used to avoid divide-by-zero errors.

2 Add visuals and test with slicer**No Department Selected (All)**

Department

☐ Finance
 ☐ HR
 ☐ IT
 ☐ Operations
 ☐ Sales

Total Headcount

40

All Employees

40

Department % of Total

100%

Department	Headcount	% of Total
Finance	8	20%
HR	6	15%
IT	8	20%
Operations	8	20%
Sales	10	25%
Total	40	100%

Sales Selected in Slicer

Department

☐ Finance

☐ HR

☐ IT

☐ Operations

☒ Sales

Total Headcount

10

All Employees

40

Department % of Total

25%

Department

Headcount

% of Total

Sales

10

25%

Total

10

25%

Key Takeaways (3B)

- ✓ Department % of Total = Department Headcount ÷ Company Headcount.
- ✓ ALL('EmployeeMaster'[Department]) removes only the department filter.
- ✓ When Sales is selected: 10 ÷ 40 = 0.25 → 25%.
- ✓ Format the measure as Percentage (0 decimal places) to show 25% (not 0.25 or 25.00%).

Very important distinction

Don't confuse these two:

ALL(EmployeeMaster)

Removes **all filters** from EmployeeMaster.

ALL('EmployeeMaster')

ALL(EmployeeMaster[Department])

Removes **only the Department filter**.

ALL('EmployeeMaster'[Department])

That's why for **Department % of Total**, we use:

ALL('EmployeeMaster'[Department])

rather than the entire table.

Explanation

CALCULATE changes filter context. ALL removes a specified filter from that context.

A simple mental model:

SLICER



Filter Context



CALCULATE



ALL()



Remove selected filter



Calculate total

Business application

This pattern is used for:

- Department contribution %
 - Sales % of company total
 - Employee % of total workforce
 - Regional contribution %
 - Expense % of total
 - Market-share calculations
-

Checkpoint question

Question:

“If I select Sales in the slicer, why does Total Headcount become 8 but All Employees remains 40?”

Expected answer:

Total Headcount respects the Department filter, while ALL(EmployeeMaster) removes the filter before calculating All Employees.

Next → Step 4: Common DAX Patterns (SUM, AVERAGE, COUNTROWS, DIVIDE).

Day 3 - Step 4: Common DAX Patterns

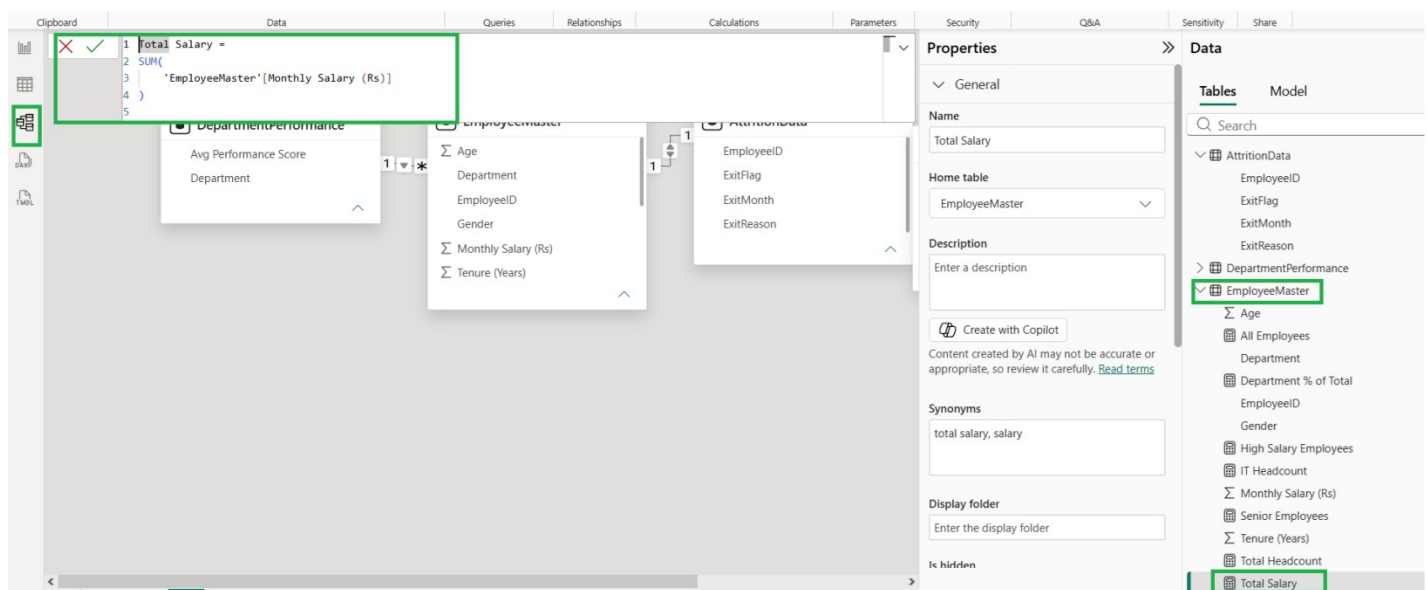
Now that participants understand **CALCULATE → FILTER → ALL**, let's introduce the most commonly used DAX functions for HR analytics.

We'll do this practically.

Step 4A — Total Salary using SUM()

Create a measure:

```
Total Salary =  
SUM(  
    'EmployeeMaster'[Monthly Salary (Rs)]  
)
```



Add a **Card**.

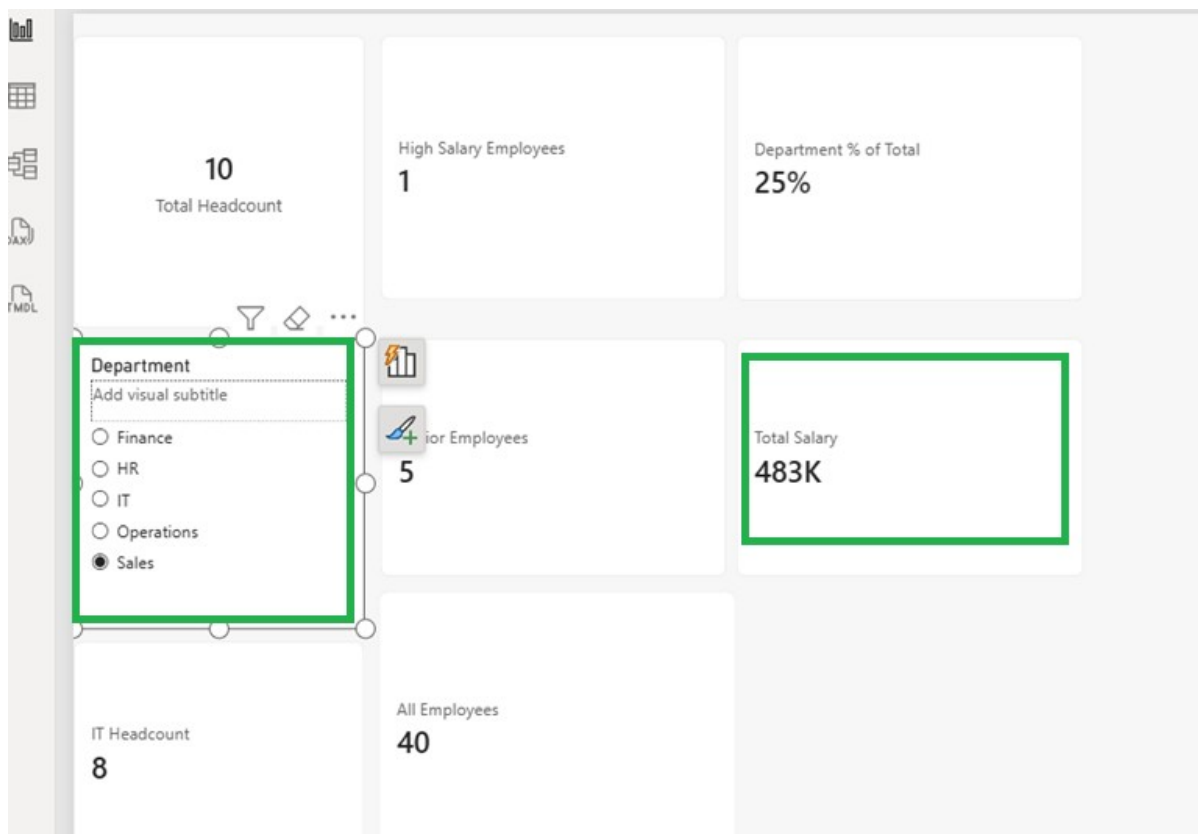
This gives the **total monthly salary cost** of all employees in the current filter context.

Test the slicer

Select **Sales**.

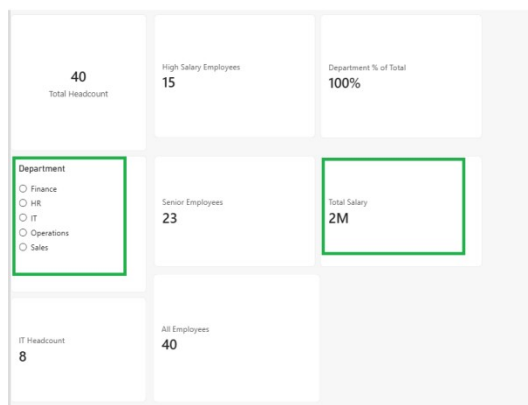
The measure now shows:

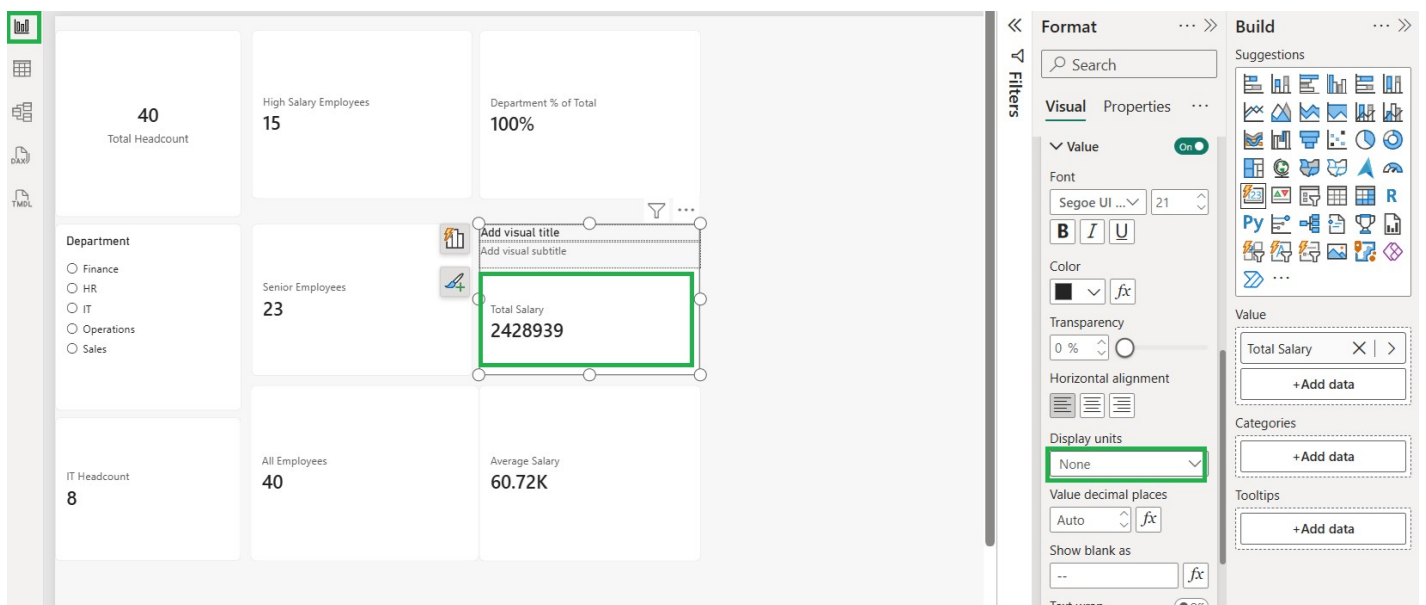
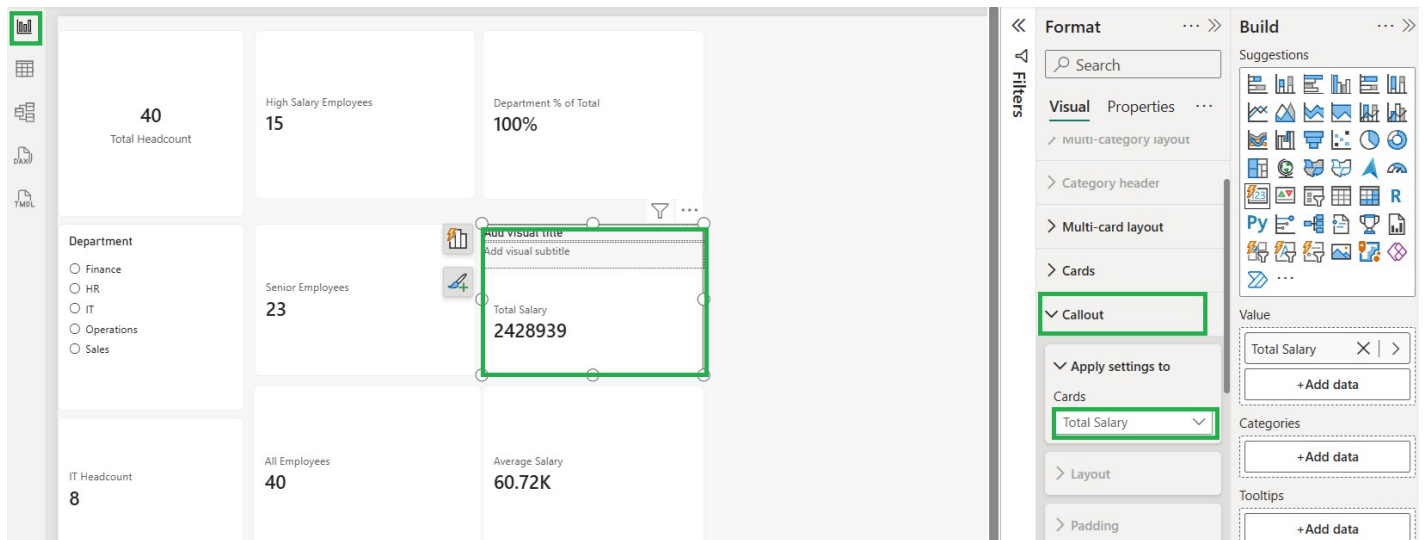
Total salary of Sales employees



Clear the slicer: **click on sales radio button**

Total salary of all employees





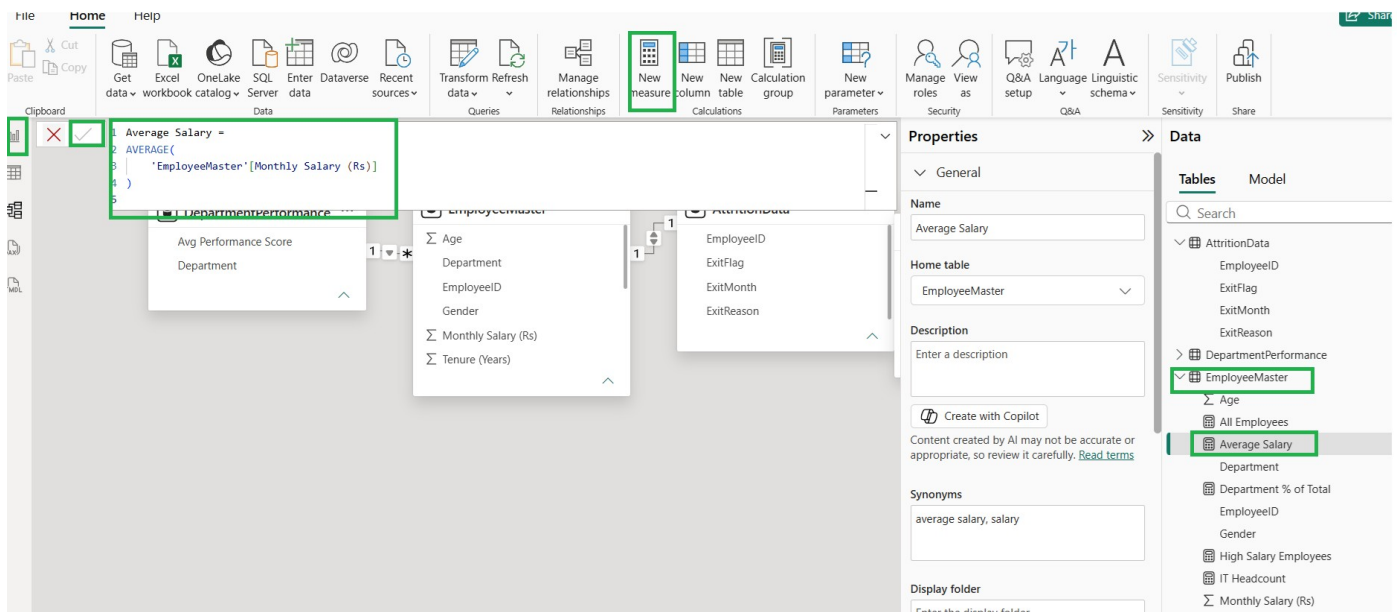
So again:

Slicer → Filter Context → Measure

Step 4B — Average Salary using AVERAGE()

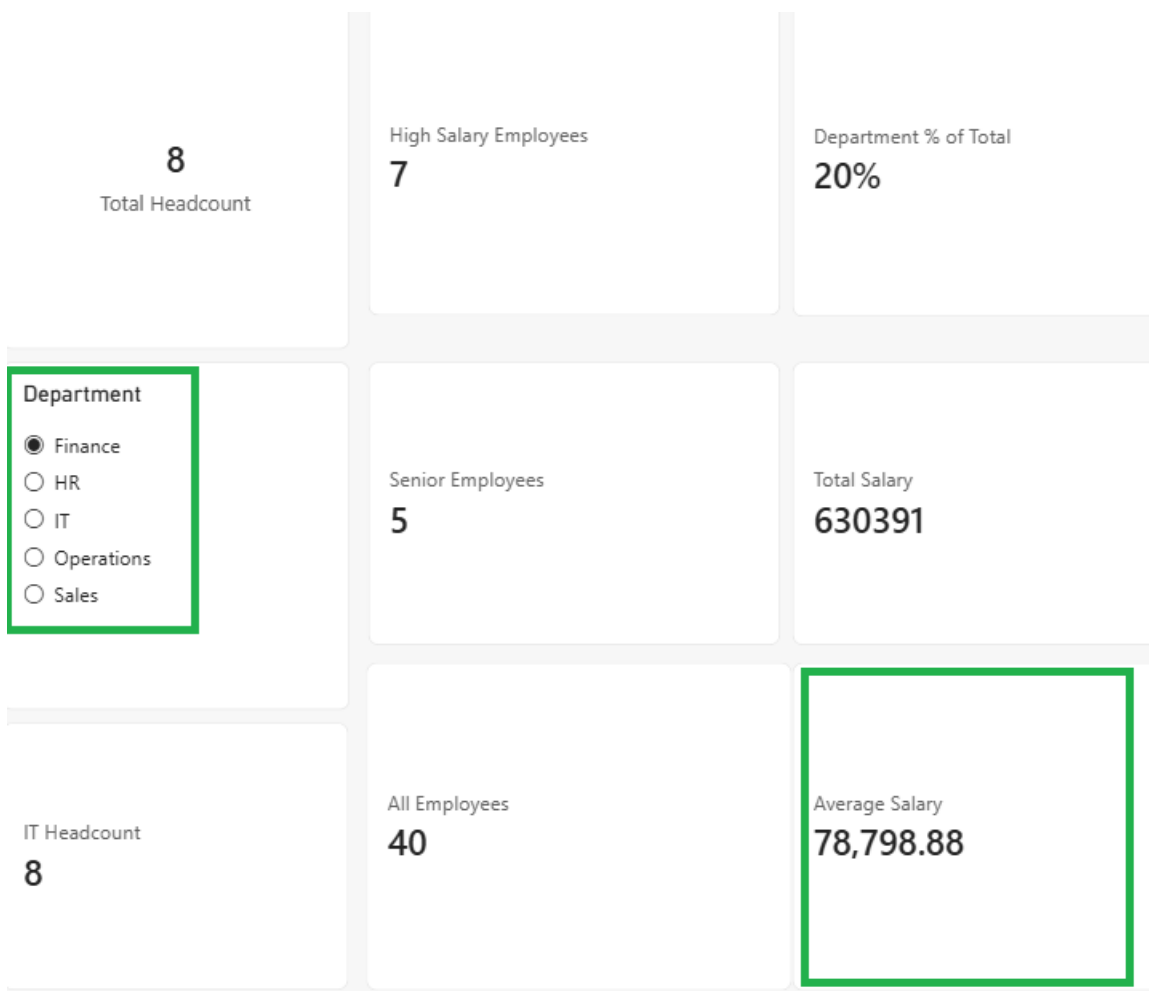
Create:

Average Salary =
 AVERAGE(
 'EmployeeMaster'[Monthly Salary (Rs)]
)



Add another Card.

Select different departments.



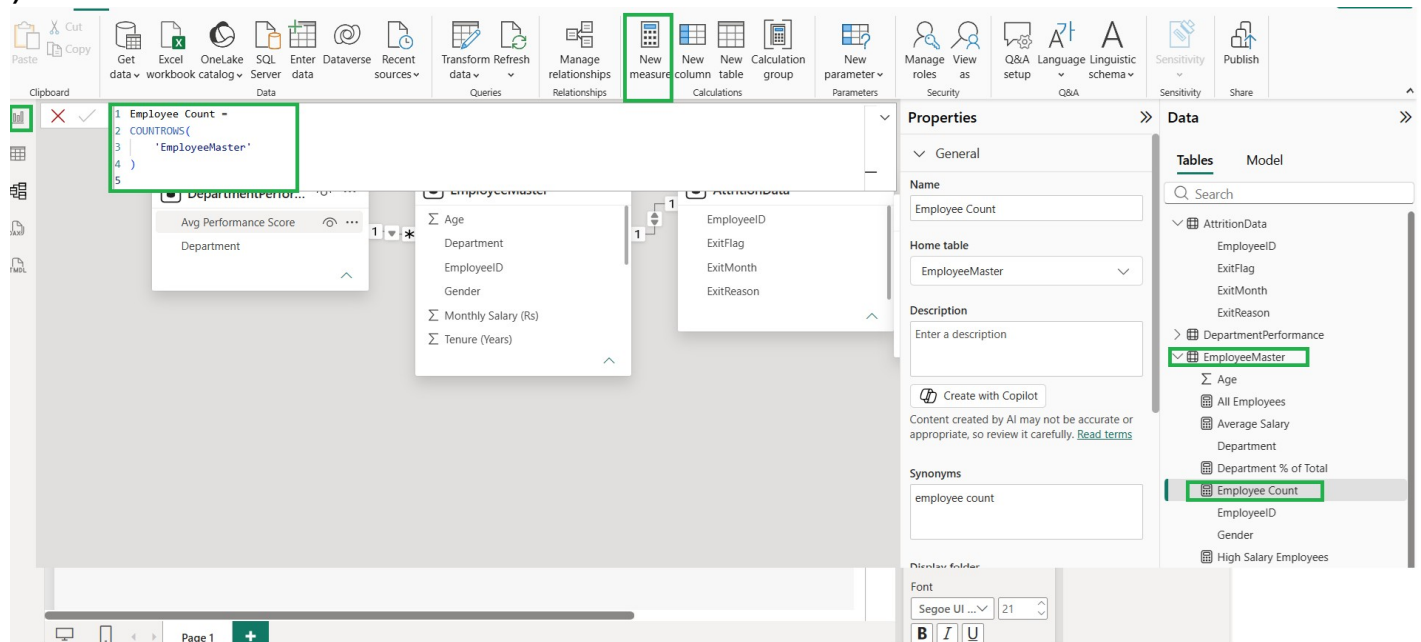
You should see the average salary change according to the selected department.

This is a very common HR KPI.

Step 4C — Count rows using COUNTROWS()

Create:

```
Employee Count =  
COUNTROWS(  
    'EmployeeMaster'  
)
```



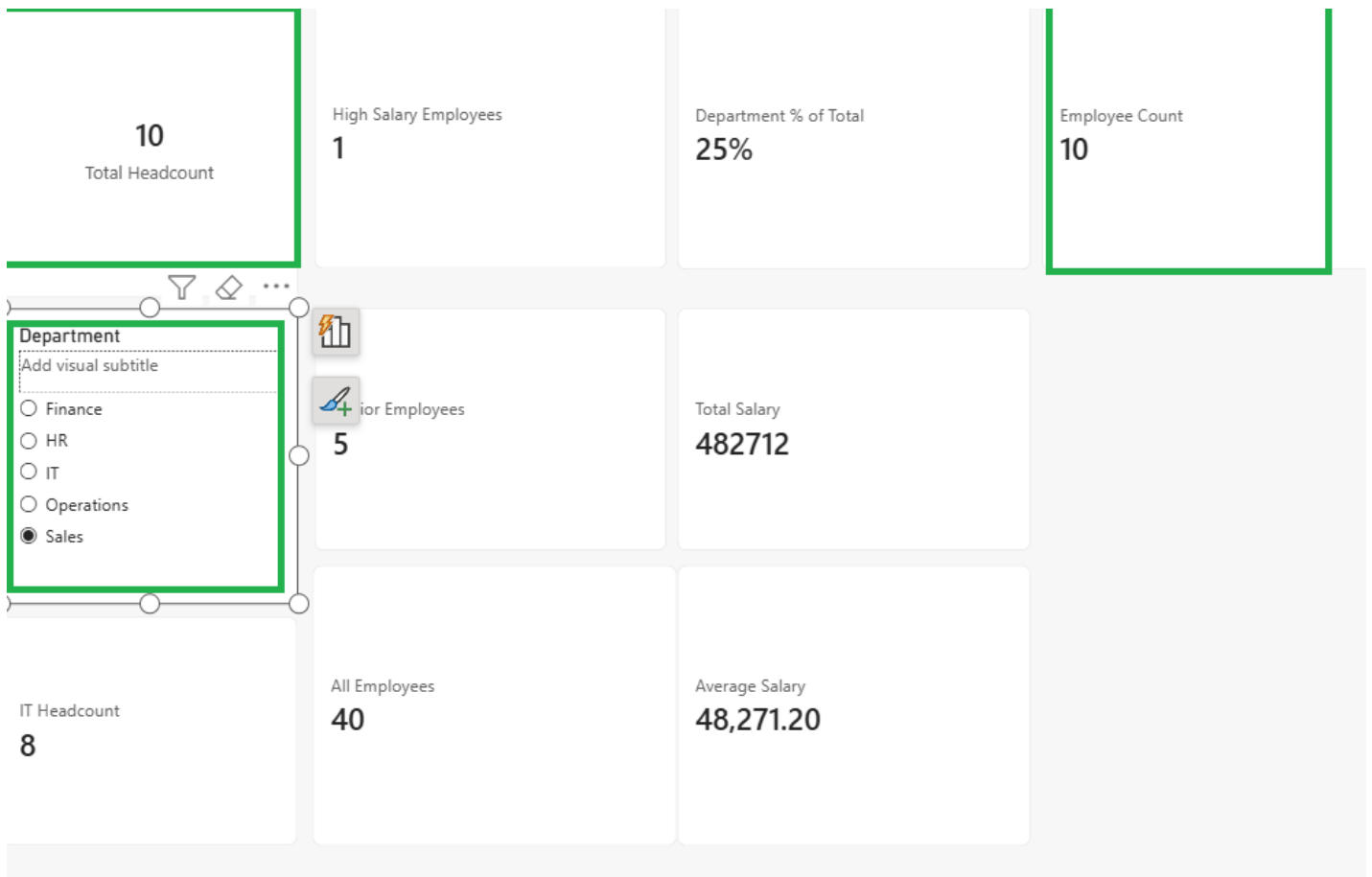
This is another way of counting employees.

Compare:

```
Total Headcount =  
COUNTA('EmployeeMaster'[EmployeeID])
```

with:

```
Employee Count =  
COUNTROWS('EmployeeMaster')
```



For your current EmployeeMaster table, both should give the same count under the same filter context.

Trainer point

COUNTROWS() is particularly useful when you want to count **rows in a table**, while COUNTA() counts non-blank values in a column.

Step 4D — DIVIDE()

You already used DIVIDE() in:

```

Department % of Total =
DIVIDE(
    [Total Headcount],
    CALCULATE(
        [Total Headcount],
        ALL('EmployeeMaster'[Department])
    ),
    0
)

```

The basic pattern is:

```
DIVIDE(  
    Numerator,  
    Denominator,  
    Alternate Result  
)
```

For example:

$$10 \div 40 = 0.25 = 25\%$$

The third argument 0 prevents an unwanted divide-by-zero error.

Step 4E — Combine the functions

Now create one practical HR measure:

```
High Salary Employees =  
CALCULATE(  
    COUNTROWS('EmployeeMaster'),  
    FILTER(  
        'EmployeeMaster',  
        'EmployeeMaster'[Monthly Salary (Rs)] > 70000  
    )  
)
```

Here we are combining:

```
COUNTROWS()  
+  
FILTER()  
+  
CALCULATE()
```

This is the type of DAX pattern participants should start recognizing.

DAX Cheat Sheet

Function	Purpose	HR Example
SUM()	Add values	Total Salary
AVERAGE()	Calculate average	Average Salary
COUNTROWS()	Count table rows	Employee Count
DIVIDE()	Safe division	Attrition Rate
CALCULATE()	Modify filter context	IT Headcount
FILTER()	Apply conditions	Salary > ₹70K

ALL()	Remove filters	Company Total
--------------	----------------	------------------

checkpoint

Ask participants:

Which function should you use?

1. Total salary → SUM()
 2. Average salary → AVERAGE()
 3. Number of employee records → COUNTROWS()
 4. Percentage → DIVIDE()
 5. Change filter context → CALCULATE()
 6. Salary > ₹70,000 → FILTER()
 7. Ignore Department slicer → ALL()
-

Next → Step 5: RANKX()

This is where Day 3 becomes more advanced: we'll rank **departments by number of employees/leaves** and identify the **Rank 1 department**.

Day 3 - Step 5: RANKX()

Now we move to an important **advanced DAX function**: ranking.

Our business question:

Which department has the highest headcount?

We will rank all departments based on **Total Headcount**.

Step 5A — Create Department Rank

Go to **Modeling** → **New measure** and create:

Department Rank =
RANKX(

```

ALL('EmployeeMaster'[Department]),
[Total Headcount],
,
DESC,
DENSE
)

```

What does this mean?

RANKX



Look at all Departments



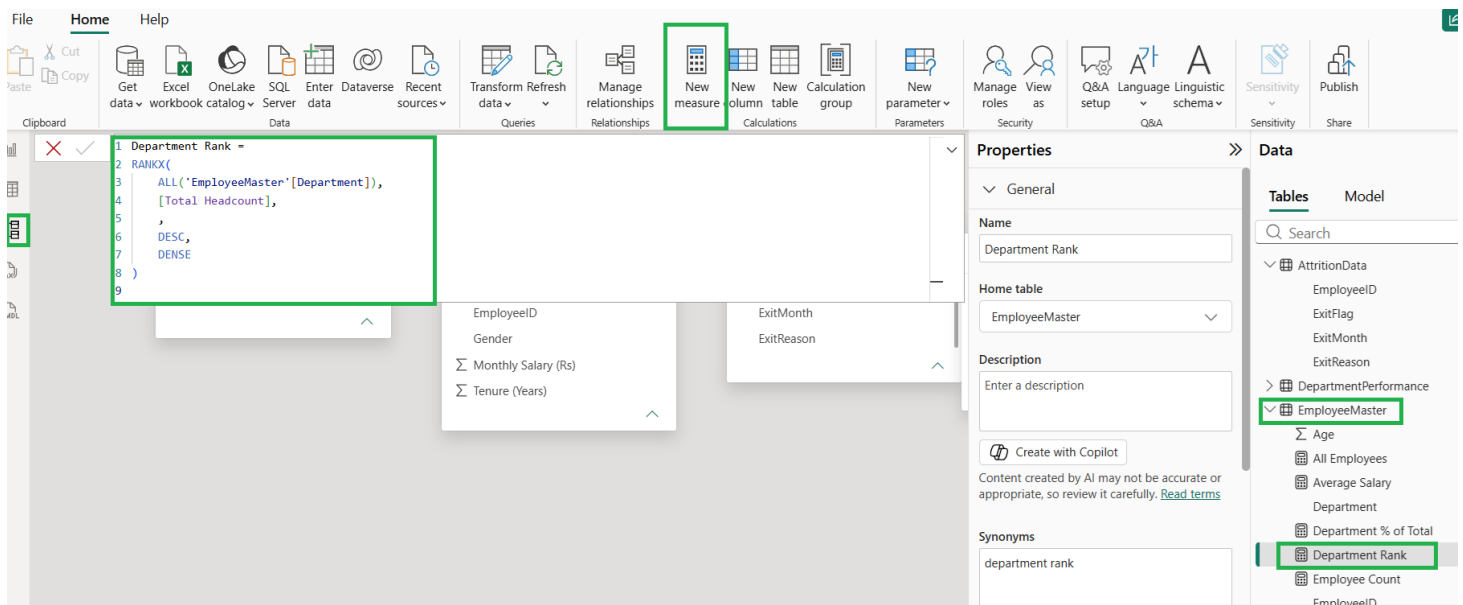
Calculate Total Headcount for each



Sort DESCENDING



Highest Headcount = Rank 1



Step 5B — Create a Table visual

Add a **Table** visual.

Put these fields into the table:

- Department
- [Total Headcount]
- [Department Rank]

You should get something similar to:

Department	Headcount	Rank
Sales	10	1
Finance	8	2
IT	8	2
Operations	8	2
HR	6	5

The exact ordering can depend on the visual sorting.

Why are Finance, IT and Operations all Rank 2?

Because they all have **8 employees**.

We used:

DENSE

So tied values receive the same rank.

Step 5C — Test the Department slicer

This is the important part.

Select **Sales** in your slicer.

You will see:

Total Headcount = 10

Department Rank = 1

Why does Rank remain **1**?

Because we used:

ALL('EmployeeMaster'[Department])

inside RANKX.

This tells Power BI:

"When calculating the ranking, consider all departments, not just the department selected in the slicer."

Compare these two concepts

ALL() in our previous step:

```
All Employees =  
CALCULATE(  
    [Total Headcount],  
    ALL('EmployeeMaster')  
)
```

→ Remove filters to get the **company total**.

ALL() inside RANKX:

```
RANKX(  
    ALL('EmployeeMaster'[Department]),  
    [Total Headcount],  
    ,  
    DESC,  
    DENSE  
)
```

Consider **all departments** when calculating the rank.

checkpoint

Ask:

Q: Why do we use ALL(Department) inside RANKX?

Expected answer:

To make Power BI compare the selected department against all departments rather than ranking it only within the current filtered context.

Key takeaway

RANKX() ranks values based on an expression, and ALL() allows us to establish the complete comparison set.

Next → Step 6: Time Intelligence

We'll create **Last Year Leaves** and **Leave Change % vs Last Year**. Before writing the exact DAX, we'll first check whether your project data

has a proper **Date/Year/Period column**, because time-intelligence functions depend on the actual date structure.

Day 3 - Step 6: Time Intelligence

Good — your current dataset **does support a Year-over-Year comparison**. The MonthlyHeadcount table has two 12-month periods: **Jan-Dec** and **Jan-Y2-Dec-Y2**.

So we can properly demonstrate **Last Year Leaves → Leave Change %**.

Step 6A — Create a proper Date column

Your Month column is text such as:

Jan
Feb
...
Dec
Jan-Y2
Feb-Y2
...
Dec-Y2

Power BI time-intelligence functions work much better with an actual **Date** column.

Go to:

Table view → MonthlyHeadcount → New column

File Home Help **Table tools** Share

Name MonthlyHeadcount

Manage relationships Relationships New measure measure column Calculations **New table** Mark as date table Calendars

Structure

Month	Opening Headcount	New Hires	Exits
Jan	50	4	2
Feb	52	3	1
Mar	54	2	3
Apr	53	5	1
May	57	3	2
Jun	58	4	3
Jul	58	4	0
Aug	62	5	1
Sep	66	5	1
Oct	70	2	0
Nov	72	5	0
Dec	77	4	0
Jan-Y2	81	4	0
Feb-Y2	85	2	0
Mar-Y2	87	5	0
Apr-Y2	92	3	1
May-Y2	94	2	0
Jun-Y2	96	5	1
Jul-Y2	100	3	1
Aug-Y2	102	5	0
Sep-Y2	107	4	1
Oct-Y2	110	5	0
Nov-Y2	115	4	1
Dec-Y2	118	3	0

Data

Search

- AttritionData
 - EmployeeID
 - ExitFlag
 - ExitMonth
 - ExitReason
- DepartmentPerformance
- EmployeeMaster
- MonthlyHeadcount**

Create:

Period Date =

VAR MonthName =

LEFT('MonthlyHeadcount'[Month], 3)

VAR YearNumber =

IF(

CONTAINSSTRING('MonthlyHeadcount'[Month], "-Y2"),
2026,
2025

)

VAR MonthNumber =

SWITCH(

MonthName,

"Jan", 1,

"Feb", 2,

"Mar", 3,

"Apr", 4,

"May", 5,

"Jun", 6,

"Jul", 7,

"Aug", 8,

"Sep", 9,

"Oct", 10,

"Nov", 11,

"Dec", 12

)

RETURN

DATE(YearNumber, MonthNumber, 1)

File Home Help Table tools **Measure tools** Share

Name: Measure 2 Format: \$% Data category: Uncategorized

Home table: MonthlyHeadcount \$ % -00 Auto

Structure Formatting Properties

1 Measure Period Date =
 2 VAR MonthName =
 3 LEFT('MonthlyHeadcount'[Month], 3)
 4 VAR YearNumber =
 5 IF(
 6 CONTAINSSTRING('MonthlyHeadcount'[Month], "-Y2"),
 7 2026,
 8 2025
 9)
 10 VAR MonthNumber =
 11 SWITCH(
 12 MonthName,
 13 "Jan", 1,
 14 "Feb", 2,
 15 "Mar", 3,
 16 "Apr", 4,
 17 "May", 5,
 18 "Jun", 6,
 19 "Jul", 7,
 20 "Aug", 8,
 21 "Sep", 9,
 22 "Oct", 10,
 23 "Nov", 11,
 24 "Dec", 12
 25)
 26 RETURN
 27 DATE(YearNumber, MonthNumber, 1)

Month Opening Headcount New Hires Exits

Month	Opening Headcount	New Hires	Exits
Jan	50	4	2
Feb	52	3	1
Mar	54	2	3
Apr	53	5	1
May	57	3	2

Table: MonthlyHeadcount (24 rows) Column: Measure 2 (0 distinct values)

Update available (click to download)

File Home Help Table tools **Column tools** Share

Name: Period Date Format: 13/14/2001 1:30:55... Summarization: Don't summarize Data category: Uncategorized

Data type: Date/time \$ % -00 Auto

Structure Formatting Properties Sort by column Data groups Manage relationships **New column Calculations**

1 Period Date =
 2 VAR MonthName =
 3 LEFT('MonthlyHeadcount'[Month], 3)
 4 VAR YearNumber =
 5 IF(
 6 CONTAINSSTRING('MonthlyHeadcount'[Month], "-Y2"),
 7 2026,
 8 2025
 9)
 10 VAR MonthNumber =
 11 SWITCH(
 12 MonthName,
 13 "Jan", 1,
 14 "Feb", 2,
 15 "Mar", 3,
 16 "Apr", 4,
 17 "May", 5,
 18 "Jun", 6,
 19 "Jul", 7,
 20 "Aug", 8,
 21 "Sep", 9,
 22 "Oct", 10,
 23 "Nov", 11,
 24 "Dec", 12
 25)
 26 RETURN
 27 DATE(YearNumber, MonthNumber, 1)
 28

Month Opening Headcount New Hires Exits Period Date

Month	Opening Headcount	New Hires	Exits	Period Date
Jan	50	4	2	1/1/2025 12:00:00 AM
Feb	52	3	1	2/1/2025 12:00:00 AM
Mar	54	2	3	3/1/2025 12:00:00 AM
Apr	53	5	1	4/1/2025 12:00:00 AM

Data

- AttritionData
 - EmployeeID
 - ExitFlag
 - ExitMonth
 - ExitReason
- DepartmentPerformance
- EmployeeMaster
- MonthlyHeadcount
 - Exits
 - Measure 2
 - Month
 - New Hires
 - Opening Headcount
- Period Date

What this creates

Your data will now have:

Month	Period Date
Jan	Jan 2025
Feb	Feb 2025

Mar	Mar 2025
...	...
Dec	Dec 2025
Jan-Y2	Jan 2026
Feb-Y2	Feb 2026
Mar-Y2	Mar 2026
...	...
Dec-Y2	Dec 2026

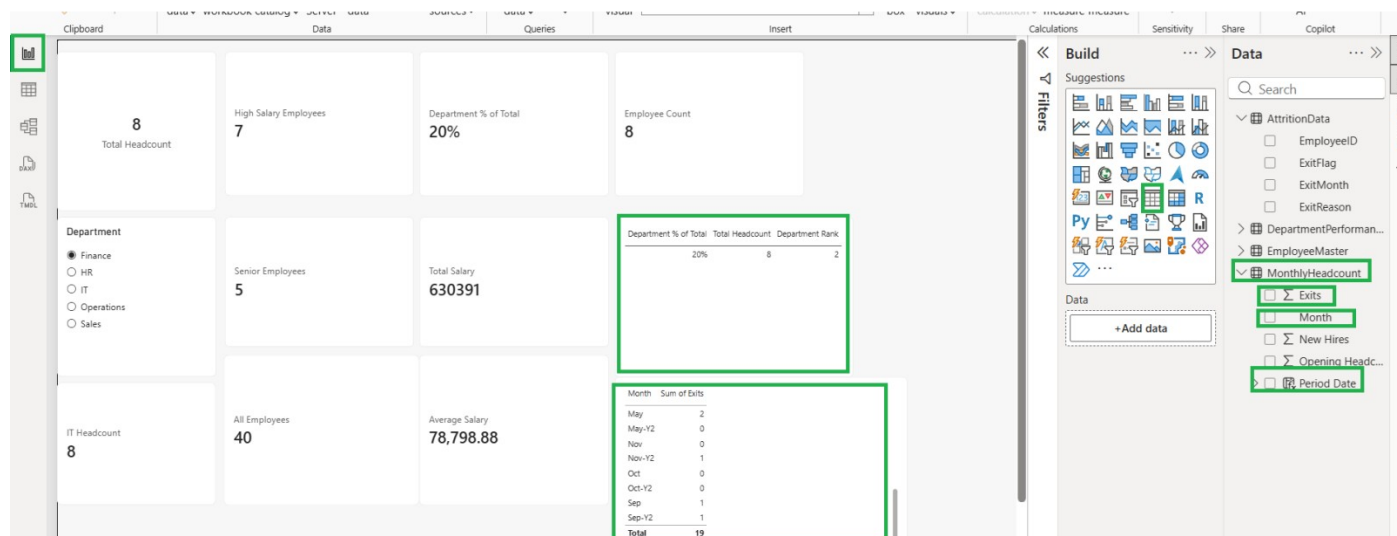
Important: The years 2025/2026 are simply being used to establish the two sequential periods in your training dataset; the source itself labels them as the first period and Y2.

Step 6B — Check the result

Add a **Table** visual with:

- Month
- Period Date
- Exits

You should see the 24 periods with proper dates.



Step 6B — Create a Date Table

We need a proper Date table before using SAMEPERIODLASTYEAR().

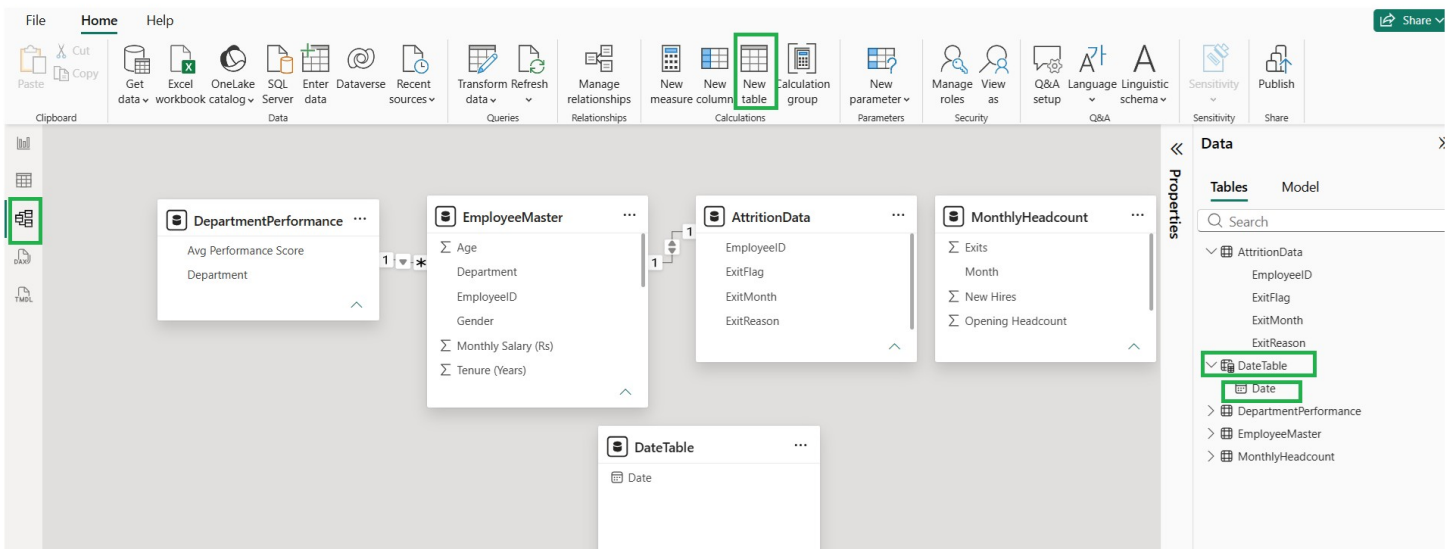
1. Go to Modeling → New table

Create:

```
DateTable =  
CALENDAR(  
    MIN('MonthlyHeadcount'[Period Date]),  
    MAX('MonthlyHeadcount'[Period Date])  
)
```

Press **Enter**.

Power BI will create a new table called **DateTable**.



2. Create a Year column

Select **DateTable** → **New column**:

```
Year =  
YEAR('DateTable'[Date])
```

3. Create a Month column

```
Month =  
FORMAT('DateTable'[Date], "MMM")
```

4. Create Month Number

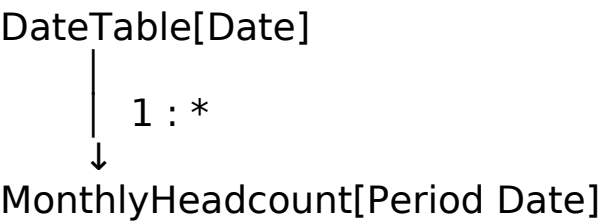
Month Number =

MONTH('DataTable'[Date])

Step 6C — Create the relationship

Go to **Model view**.

Connect:



The screenshot shows the 'New relationship' dialog box in Power BI Desktop. The 'From table' is 'DataTable' and the 'To table' is 'MonthlyHeadcount'. The 'Date' column from 'DataTable' is selected as the relationship column, and the 'Period Date' column from 'MonthlyHeadcount' is selected as the relationship column. The cardinality is set to 'One to many (1:*)' and the cross-filter direction is set to 'Single'. The 'Make this relationship active' checkbox is checked.

Date	Month	Month Number	Year
1/1/2025 12:00:00	Jan	1	2025
1/2/2025 12:00:00	Jan	1	2025
1/3/2025 12:00:00	Jan	1	2025

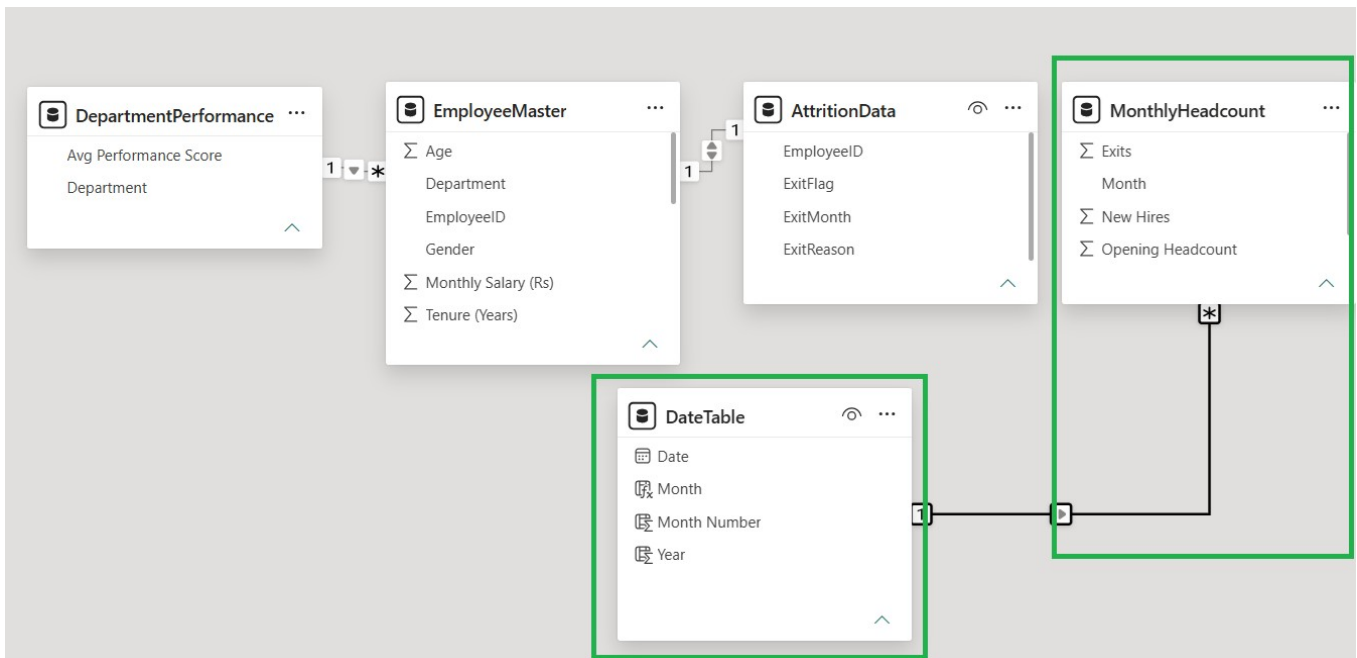
Exits	Month	New Hires	Opening Hires	Period Date
2	Jan	4	50	1/1/2025 12:00:00
1	Feb	3	52	2/1/2025 12:00:00
3	Mar	2	54	3/1/2025 12:00:00

Cardinality: One to many (1:*)

Cross-filter direction: Single

☒ Make this relationship active

☐ Apply security filter in both directions



Step 6D — Create Total Leaves/Exits

In your dataset the column is actually called **Exits**, so let's use that rather than calling it Leaves.

Create:

Total Leaves =
`SUM('MonthlyHeadcount'[Exits])`

Your project requirement calls this **number of leaves**, while the source table uses the column **Exits**.

Step 6E — Create Last Year Leaves

Now create:

Last Year Leaves =
`CALCULATE(
 [Total Leaves],
 SAMEPERIODLASTYEAR('DateTable'[Date])
)`

Step 6F — Create Leave Change %

Finally:

Leave Change % =
 DIVIDE(
 [Total Leaves] - [Last Year Leaves],
 [Last Year Leaves],
 0
)

Format it as:

Percentage → 0 decimal places

Day 3 – Step 6: Time Intelligence (Last Year Leaves & Leave Change %)

We have a Period Date column in MonthlyHeadcount. Now we will create a Date table, relate it, and build measures for Last Year Leaves and Leave Change %.

1 Create Date Table

File Home Insert **Modeling** View Optimize Help

New measure Quick measure **New table** New column New parameter What if

Calculations

```

DateTable =
CALENDAR(
  MIN('MonthlyHeadcount'[Period Date]),
  MAX('MonthlyHeadcount'[Period Date])
)

```

2 Add Date related columns

```

Year =
YEAR('DateTable'[Date])

Month =
FORMAT('DateTable'[Date], "MMM")

Month Number =
MONTH('DateTable'[Date])

```

Date	Year	Month	Month Number
01-01-2025	2025	Jan	1
01-02-2025	2025	Feb	2
01-03-2025	2025	Mar	3
...			
01-12-2026	2026	Dec	12

3 Create Relationship

Create a 1:* relationship:
DateTable[Date] → MonthlyHeadcount[Period Date]

4 Create Total Leaves Measure

```

Total Leaves =
SUM('MonthlyHeadcount'[Exits])

```

5 Create Last Year Leaves Measure

```

Last Year Leaves =
CALCULATE(
  [Total Leaves],
  SAMEPERIODLASTYEAR('DateTable'[Date])
)

```

6 Create Leave Change % Measure

```

Leave Change % =
DIVIDE(
  [Total Leaves] - [Last Year Leaves],
  [Last Year Leaves],
  0
)

```

Format this measure as Percentage (0 decimal places).

7 Visualize the Result

Build a Table visual

- Add DateTable[Year]
- Add DateTable[Month]
- Add [Total Leaves]
- Add [Last Year Leaves]
- Add [Leave Change %]

Year	Month	Total Leaves	Last Year Leaves	Leave Change %
2025	Jan	2	-	-
2025	Feb	1	-	-
2025	Mar	1	-	-
2025	Apr	2	-	-
2025	May	3	-	-
2025	Jun	2	-	-
2025	Jul	2	-	-
2025	Aug	2	-	-
2025	Sep	1	-	-
2025	Oct	2	-	-
2025	Nov	3	-	-
2025	Dec	2	-	-
2026	Jan	3	2	50%
2026	Feb	2	1	100%
2026	Mar	3	1	200%
2026	Apr	4	2	100%
2026	May	3	3	0%
2026	Jun	2	2	0%

Total Leaves vs Last Year Leaves by Month

Key Observations

- 2025 has no Last Year Leaves (so blank).
- For 2026, Last Year Leaves shows the value from the same month in 2025.
- Leave Change % shows increase/decrease vs last year.
- This uses SAMEPERIODLASTYEAR() with DateTable[Date].